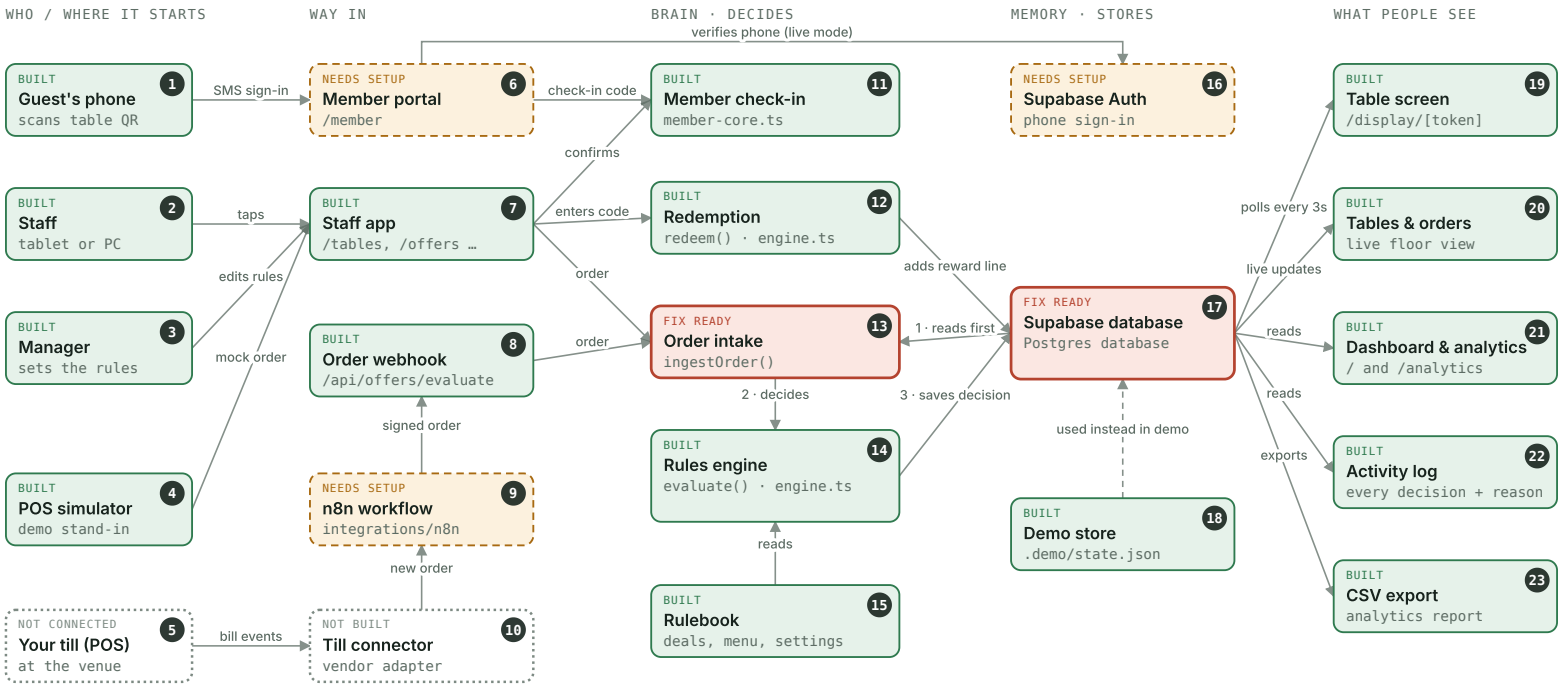
TABLEPULSE · THE SUNDAY SOCIAL · AS OF 9 OCT 2026

How TablePulse fits together

Every part of the system, how orders and decisions flow between them, and what state each one is in. Use it to see what's finished and where a new feature would plug in.

Built works today **Needs setup** code done, needs an account or config **Fix ready** works, but has a bug with a fix in `sim/fixes.patch`

Not built doesn't exist yet



Orders reach the brain two ways: staff or the simulator through the staff app (works now), or your real till through a connector and n8n (connector not built). For every order, TablePulse first loads the current state from the database (tables, orders, stock, customer history), then the rules engine decides, then the result is saved back in one step. The screens read from the database. Dashed arrow: demo mode swaps the database for a local file. Click any box, or read every box explained.

Every box explained

Numbers match the map. Click any box on the map to jump to its note, and click a name under “Gets from” or “Sends to” to follow the arrow.

WHO / WHERE IT STARTS

1 Guest's phone — BUILT

The guest’s own phone. They scan the QR code on the table screen to join as a member or check in for this visit.

Status: Works today. In demo mode the SMS code is shown on screen instead of being texted.

GETS FROM	SENDS TO
starts here	**6. Member portal** SMS sign-in

Lives in: `Nothing to install. It opens the /member page in the phone’s browser.`

2 Staff — BUILT

Servers and bar staff, on a venue tablet or PC. They open tables, tick the 18+ box, link guests and enter reward codes.

Status: Works today. Staff can only see and do what their role allows.

GETS FROM	SENDS TO
starts here	**7. Staff app** taps

Lives in: `Roles: staff, manager, platform admin.`

3 Manager — BUILT

The venue manager. Sets the deals, menu costs and stock, and the profit and score limits the engine must respect.

Status: Works today. Only managers can change settings and see customer data.

GETS FROM	SENDS TO
starts here	**7. Staff app** edits rules

Lives in: `Uses the same staff app, with more pages unlocked.`

4 POS simulator — BUILT

A page in the staff app that pretends to be a till. You pick a table and items and press send, and the order goes through the real engine.

Status: Works today, for demos and testing. Not used in live service.

GETS FROM	SENDS TO
starts here	**7. Staff app** mock order

Lives in: `app/(workspace)/simulator · components/simulator.tsx`

5 Your till (POS) — NOT CONNECTED

The venue’s point-of-sale system, where orders are normally rung up and paid. In live service it should tell TablePulse about each new order automatically.

Status: Not connected. TablePulse doesn’t know which POS you use yet, so for now orders must be typed into the staff app.

GETS FROM	SENDS TO
starts here	**10. Till connector** bill events

Lives in: `Your existing POS. TablePulse never takes payment.`

WAY IN

6 Member portal — NEEDS SETUP

The guest’s membership page. They sign in with a phone number and SMS code, get a membership number, choose optional preferences, and request a table check-in code.

Status: Code is finished. Real SMS needs a Supabase project with a phone/SMS provider and an hCaptcha key. Demo mode shows the code on screen.

GETS FROM	SENDS TO
1. Guest's phone SMS sign-in	**11. Member check-in** check-in code
16. Supabase Auth confirms the code	**16. Supabase Auth** verifies phone (live mode)

Lives in: `app/member · components/member-portal.tsx · lib/server/members.ts`

7 Staff app — BUILT

The web app staff and managers use: overview, tables and orders, offers, menu, analytics, activity log, settings and the simulator.

Status: Works today. Every action is checked against the user's role on the server.

GETS FROM	SENDS TO
2. Staff taps	**13. Order intake** order
3. Manager edits rules	**12. Redemption** enters code
4. POS simulator mock order	**11. Member check-in** confirms

Lives in: `app/(workspace) · components/*.tsx · lib/server/commands.ts`

8 Order webhook — BUILT

The secure address where outside systems send orders. Each request must be signed with a secret and be less than 5 minutes old, so nobody can fake orders.

Status: Works today. It's waiting for n8n or a till connector to send to it.

GETS FROM	SENDS TO
9. n8n workflow signed order	**13. Order intake** order

Lives in: `app/api/[...path]/route.ts (/api/offers/evaluate and /api/pos/events)`

9 n8n workflow — NEEDS SETUP

An automation workflow that sits between the till and TablePulse. It checks the order is well-formed, signs it, sends it, and tells the till the result (shown, withheld, control or duplicate). It makes no decisions itself.

Status: Built and tested here, but not yet imported into your n8n account. Needs two keys and your app's web address.

GETS FROM	SENDS TO
10. Till connector new order	**8. Order webhook** signed order

Lives in: `integrations/n8n/tablepulse-pos-decision.json`

10 Till connector — NOT BUILT

A small translator written for your specific POS brand. It turns the till's bills and products into TablePulse visits and menu items, and handles voids, split bills and table moves.

Status: Not built. Depends on which POS you use and what data it lets other apps read.

GETS FROM	SENDS TO
5. Your till (POS) bill events	**9. n8n workflow** new order

Lives in: `lib/pos/adapter.ts defines the shape it must follow.`

BRAIN · DECIDES

11 Member check-in — BUILT

Links a signed-in member to the table they're actually sitting at. The guest's phone shows a 6-digit code, and the server types it in to confirm. Codes last 5 minutes, work once, and only for that table.

Status: Works today. Checking in never creates a reward by itself.

GETS FROM	SENDS TO
6. Member portal check-in code	**17. Supabase database** saves the guest–visit link
7. Staff app confirms	

Lives in: `lib/member-core.ts`

12 Redemption — BUILT

Runs when a guest accepts a deal and the server enters the 6-character code. It re-checks every rule, then adds the reward as a separate line on the bill, takes it out of stock and records revenue and profit.

Status: Works today. 5 wrong codes lock that table's redemption for 5 minutes.

GETS FROM	SENDS TO
7. Staff app enters code	**17. Supabase database** adds reward line

Lives in: `redeem() in lib/engine.ts`

13 Order intake — FIX READY

Receives every new order, from staff or the till. It ignores repeats, checks stock, takes items out of stock, saves the order and then asks the rules engine to decide.

Status: Works, with one bug: when a till re-sends the same order, it's rejected as a conflict instead of being recognised as a repeat. Fix ready in sim/fixes.patch.

GETS FROM	SENDS TO
8. Order webhook order	**14. Rules engine** 2 · decides
7. Staff app order	
17. Supabase database 1 · reads first	

Lives in: `ingestOrder() in lib/server/commands.ts`

14 Rules engine — BUILT

The brain. For each new order it checks whether the table can get a deal at all, finds deals that fit the basket, removes any that break stock, safety, limits or profit rules, scores the rest, and shows the best one 78% of the time. It writes down a reason either way.

Status: Works today. Deals only look at the basket, not party size, zone or time.

GETS FROM	SENDS TO
13. Order intake 2 · decides	**17. Supabase database** 3 · saves decision
15. Rulebook reads	

Lives in: `evaluate() and matches() in lib/engine.ts`

15 Rulebook — BUILT

Everything the engine reads before deciding: the 5 deals and their prices and limits, menu costs and stock, and the venue settings (S$2.50 profit floor, score 65, 78% chance, 15% control).

Status: Works today. Changeable in the app, except new kinds of deal, which need code.

GETS FROM	SENDS TO
starts here	**14. Rules engine** reads

Lives in: `Offers, Menu and Settings pages · starting values in lib/seed.ts`

MEMORY · STORES

16 Supabase Auth — NEEDS SETUP

Supabase's login service. It sends and checks guests' SMS codes and staff email logins, and keeps phone numbers out of TablePulse's own tables.

Status: Not set up. Needs a Supabase project with phone sign-in, an SMS provider and hCaptcha turned on.

GETS FROM	SENDS TO
6. Member portal verifies phone (live mode)	**6. Member portal** confirms the code

Lives in: `Configured in your Supabase project, not in code.`

17 Supabase database — FIX READY

Where everything is stored: tables, visits, orders, menu, stock, deals, decisions, customers, reward history and the audit log. Read first on every order, then written back in one step. Access rules stop staff from seeing private customer data.

Status: Setup files are ready but no Supabase project is connected yet. One bug: busy tables can make orders fail. Fix ready in sim/fixes.patch.

GETS FROM	SENDS TO
14. Rules engine 3 · saves decision	**13. Order intake** 1 · reads first
12. Redemption adds reward line	**19. Table screen** polls every 3s
18. Demo store used instead in demo	**20. Tables & orders** live updates
11. Member check-in saves the guest–visit link	**21. Dashboard & analytics** reads
	22. Activity log reads
	23. CSV export exports

Lives in: `supabase/migrations/*.sql · lib/server/store.ts`

18 Demo store — BUILT

A simple file on the computer that replaces the database in demo mode, so you can try everything without a Supabase account.

Status: Works today. For one computer only, not for live service.

GETS FROM	SENDS TO
starts here	**17. Supabase database** used instead in demo

Lives in: `.demo/state.json · reset from the POS simulator page`

WHAT PEOPLE SEE

19 Table screen — BUILT

The tablet on each table. Shows the menu and a QR code normally, and the deal, price, code and countdown when a reward is unlocked. Never shows names or history.

Status: Works today. It uses a secret link for that visit only, which stops working when the bill is closed.

GETS FROM	SENDS TO
17. Supabase database polls every 3s	end point, people look at it

Lives in: `app/display/[token]` · `components/public-screen.tsx`

20 Tables & orders — BUILT

The staff view of every table: open, dining, reward active or redeemed. Click a table to see its bill, safety checkboxes, reward code box and close button.

Status: Works today. Updates live when connected to Supabase.

GETS FROM	SENDS TO
17. Supabase database live updates	end point, people look at it

Lives in: `components/tables.tsx`

21 Dashboard & analytics — BUILT

The manager's numbers: active tables, active deals, reward revenue, estimated extra profit, conversion by deal type, and offer tables compared with control tables.

Status: Works today. The profit figure assumes the guest wouldn't have bought the item anyway.

GETS FROM	SENDS TO
17. Supabase database reads	end point, people look at it

Lives in: `components/dashboard.tsx` · `components/management.tsx`

22 Activity log — BUILT

Every decision the engine made, with all its reasons (including when no deal was shown), plus every staff action such as voids, overrides and rejected codes.

Status: Works today.

GETS FROM	SENDS TO
17. Supabase database reads	end point, people look at it

Lives in: `/activity`

23 CSV export — BUILT

Downloads the analytics as a spreadsheet file for reporting or accounting.

Status: Works today.

GETS FROM	SENDS TO
17. Supabase database exports	end point, people look at it

Lives in: `Export report button on /analytics`

Status at a glance

“How you change it” says whether a change is a setting anyone can make in the app, or a code change.

PART	STATUS	WHAT IT DOES	WHERE IT LIVES	HOW YOU CHANGE IT
Rules engine	BUILT	Decides which table gets which deal, with a written reason	`lib/engine.ts`	Thresholds in Settings; deal rules are code
Rulebook: deals	BUILT	5 deals with price, daily limit, priority, on/off	`Flash offers page · lib/seed.ts`	Price/limit/priority in the app; new deal types are code
Rulebook: menu	BUILT	Item prices, costs, stock, promotion on/off	`Menu & inventory page`	In the app
Rulebook: settings	BUILT	Profit floor, score cut-off, 78% chance, 15% control, pause	`Venue settings page`	In the app
Order intake	FIX READY	Saves orders, checks stock, ignores repeats	`ingestOrder() · lib/server/commands.ts`	Fix: re-sent orders wrongly rejected
Redemption	BUILT	Staff enters the code; re-checks everything; adds reward to bill	`redeem() · lib/engine.ts`	Code
Staff app	BUILT	Tables, offers, menu, analytics, activity, settings, simulator	`app/ · components/`	Code (look and layout)
Table screen	BUILT	Shows the deal, code and countdown; QR to join	`/display/[token] · public-screen.tsx`	Code (wording, design)
Member portal	NEEDS SETUP	Guest SMS sign-in, membership number, preferences, check-in	`/member · lib/server/members.ts`	Live SMS needs a Supabase phone provider + hCaptcha
Member check-in	BUILT	6-digit code the server confirms at the table	`lib/member-core.ts`	Code
Supabase database	FIX READY	Stores everything, with per-role access rules	`supabase/migrations/`	Needs a hosted project; fix: busy tables fail orders
Demo store	BUILT	Local file so you can try it without Supabase	`.demo/state.json`	Reset from the POS simulator page
Order webhook	BUILT	Signed entry point for orders from n8n or a till	`/api/offers/evaluate`	Code
n8n workflow	NEEDS SETUP	Passes till orders to TablePulse and returns the result	`integrations/n8n/`	Import into n8n, add 2 credentials, set the URL
Till connector	NOT BUILT	Translates your till’s bills and items into TablePulse orders	`lib/pos/adapter.ts (interface only)`	Depends on your POS vendor’s API

PART	STATUS	WHAT IT DOES	WHERE IT LIVES	HOW YOU CHANGE IT
Analytics	BUILT	Revenue, profit, conversion by deal, control comparison, CSV	`/analytics`	Code
Hosting	NOT BUILT	A public HTTPS address for the app (needed by n8n and phones)	`Dockerfile ready`	Choose a host and deploy

Ideas you might add

None of these exist yet. Effort is a rough guide: small is a day or two of work, medium about a week, large several weeks.

IDEA	PLUGS INTO	EFFORT	WHAT IT WOULD TAKE
Deals by party size, zone or time	Rules engine	`Small–medium`	Tables already store seats and zone. Add conditions in matches() and fields on each deal, e.g. “terrace only” or “Fri 5–7pm”.
Fix the bottle deal’s name check	Rules engine	`Small`	Replace the “ribeye or pasta” name match with a “pairs with wine” flag on menu items.
Deal builder (no code)	Rulebook + engine	`Medium–large`	A form on the Offers page: if basket has X and not Y, offer Z at price P. Needs new database fields and a generic matcher.
Manager alerts and daily summary	n8n	`Small–medium`	n8n flow on a schedule or event: low stock, repeated code guessing, end-of-night totals by WhatsApp or email.
Automatic “welcome back” rewards	Members + engine	`Small–medium`	Give a member a return reward after a qualifying visit. Today it only exists in the demo data.
Real till connection	Till connector	`Medium–large`	Write the adapter for your POS: map bills to visits and products to menu items, handle voids, splits and table moves.
Staff invites and roles in the app	Staff app	`Medium`	Today staff accounts are added in Supabase by hand.
Member account recovery	Member portal	`Medium`	A self-service way to move an account to a new phone number.
GST and service charge in analytics	Analytics	`Small`	Prices are treated as excluding tax and service charge today.
Learned recommendations	Rules engine	`Large`	Pick the deal most likely to add profit using visit history. Needs months of real data and control-group results first.
Multiple venues	Database + staff app	`Large`	The tables already carry a venue ID. Needs venue switching and shared member identity.

Out of scope by design: taking payments. Your till stays in charge of the bill and settlement, and TablePulse only adds a reward line.